Nama : Wafiq Nur Azizah

NIM : 2311102270

Kelas : IF-11-03

MODUL 14

1. Praktikum

   a. Code character_model.dart

```
// Nama: Wafiq Nur Azizah
// NIM: 2311102270

class Character {
  final int id;
  final String name;
  final String username;

  Character({required this.id, required this.name, required
this.username});

  factory Character.fromMap(Map<String, dynamic> map) {
    return Character(
      id: map['id'],
      name: map['name'],
      username: map['username'] ?? 'Unknown Username',
    );
  }

  Map<String, dynamic> toMap() {
    return {'id': id, 'name': name, 'username': username};
  }
}
```

   b. Code db_helper.dart

```
// Nama: Wafiq Nur Azizah
// NIM: 2311102270

import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';
import 'character_model.dart';

class DatabaseHelper {
  static final DatabaseHelper instance = DatabaseHelper._init();
  static Database? _database;

  DatabaseHelper._init();

  Future<Database> get database async {
    if (_database != null) return _database!;
```

```
    _database = await _initDB('characters.db');
    return _database!;
  }

  Future<Database> _initDB(String filePath) async {
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, filePath);
    return await openDatabase(path, version: 1, onCreate:
_createDB);
  }

  Future _createDB(Database db, int version) async {
    await db.execute('''
      CREATE TABLE characters (
        id INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        username TEXT NOT NULL
      )
    ''');
  }

  Future<void> insertCharacter(Character character) async {
    final db = await instance.database;
    await db.insert(
      'characters',
      character.toMap(),
      conflictAlgorithm: ConflictAlgorithm.replace,
    );
  }

  Future<List<Character>> getAllCharacters() async {
    final db = await instance.database;
    final maps = await db.query('characters');
    return maps.map((map) => Character.fromMap(map)).toList();
  }

  Future<void> clearTable() async {
    final db = await instance.database;
    await db.delete('characters');
  }
}
```

c. Code main.dart (Versi 1 - Demo SQLite & HTTP)

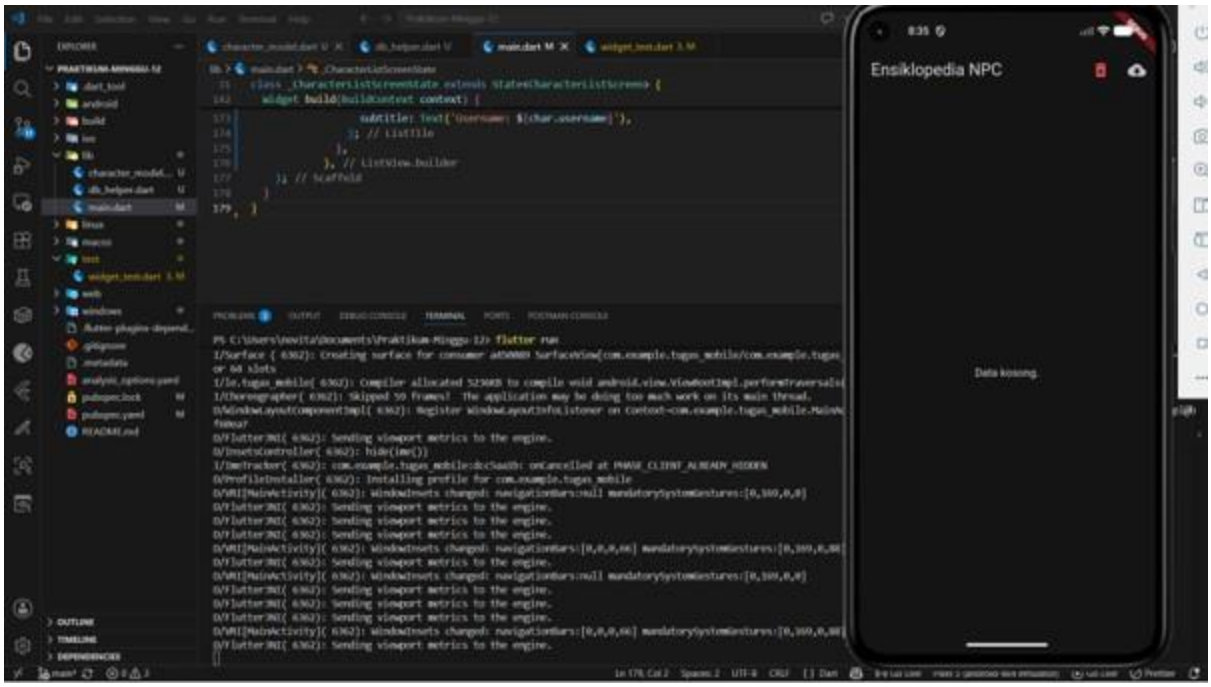
```
// Nama: Wafiq Nur Azizah
// NIM: 2311102270

import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'character_model.dart';
import 'db_helper.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
```

```
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'SQLite & HTTP Demo',
      theme: ThemeData.dark(useMaterial3: true),
      home: const CharacterListScreen(),
    );
  }
}

class CharacterListScreen extends StatefulWidget {
  const CharacterListScreen({super.key});

  @override
  State<CharacterListScreen> createState() =>
CharacterListScreenState();
}

class CharacterListScreenState extends
State<CharacterListScreen> {
  List<Character> _characters = [];
  bool _isLoading = false;

  @override
  void initState() {
    super.initState();
    _loadFromLocal();
  }

  Future<void> _loadFromLocal() async {
    setState(() => _isLoading = true);
    final localData = await
DatabaseHelper.instance.getAllCharacters();
    setState(() {
      _characters = localData;
      _isLoading = false;
    });
  }

  // Fungsi Menghapus Semua Data dari SQLite
  Future<void> _deleteAllData() async {
    bool? confirm = await showDialog<bool>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('Hapus Semua Data?'),
        content: const Text('apakah anda yakin?'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context, false),
            child: const Text('Batal'),
          ),
          TextButton(
            onPressed: () => Navigator.pop(context, true),
            child: const Text('Hapus', style: TextStyle(color:
Colors.red)),
```

```
          ),
        ],
      ),
    );

    if (confirm != true) return;

    setState(() => _isLoading = true);
    try {
      await DatabaseHelper.instance.clearTable();
      setState(() {
        _characters.clear();
      });
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Semua data berhasil
dihapus dari sistem.')),
        );
      }
    } catch (e) {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text('Terjadi kesalahan: $e')),
        );
      }
    } finally {
      setState(() => _isLoading = false);
    }
  }

  Future<void> _fetchFromAPI() async {
    setState(() => _isLoading = true);
    try {
      final response = await http.get(
        Uri.parse('https://jsonplaceholder.typicode.com/users'),
      );

      if (response.statusCode == 200) {
        final List<dynamic> dataJson =
json.decode(response.body);
        await DatabaseHelper.instance.clearTable();

        List<Character> freshCharacters = [];
        for (var item in dataJson) {
          final char = Character.fromMap(item);
          freshCharacters.add(char);
          await DatabaseHelper.instance.insertCharacter(char);
        }
        setState(() => _characters = freshCharacters);

        if (mounted) {
          ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('Berhasil sync data
dari server!')),
          );
        }
      }
```

```
    } catch (e) {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Gagal konek internet,
pastikan online.')),
        );
      }
    } finally {
      setState(() => _isLoading = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Ensiklopedia NPC'),
        actions: [
          IconButton(
            icon: const Icon(Icons.delete_forever, color:
Colors.redAccent),
            tooltip: 'Hapus Semua Data',
            onPressed: _characters.isEmpty ? null :
_deleteAllData,
          ),
          IconButton(
            icon: const Icon(Icons.cloud_download),
            tooltip: 'Sync dari Server',
            onPressed: _fetchFromAPI,
          )
        ],
      ),
      body: _isLoading
          ? const Center(child: CircularProgressIndicator())
          : _characters.isEmpty
              ? const Center(child: Text('Data kosong.'))
              : ListView.builder(
                  itemCount: _characters.length,
                  itemBuilder: (context, index) {
                    final char = _characters[index];
                    return ListTile(
                      leading: CircleAvatar(
                        backgroundColor: Colors.deepPurple,
                        child: Text(char.id.toString()),
                      ),
                      title: Text(char.name),
                      subtitle: Text('Username:
${char.username}'),
                    );
                  },
                ),
    );
  }
}
```

d. Screenshot output



e. Penjelasan dari code dan output (korelasi)

Dari ketiga source code di atas saling terintegrasi untuk membangun sebuah aplikasi Flutter yang menyoroti proses sinkronisasi antara server (REST API) dan penyimpanan lokal (SQLite). Arsitektur aplikasi ini terbagi ke dalam tiga komponen pokok, yaitu character_model.dart bertugas merancang kerangka objek dan melakukan parsing data, db_helper.dart bertanggung jawab atas seluruh manajemen database internal (seperti koneksi, pembuatan tabel, dan operasi CRUD), serta main.dart yang bertindak sebagai antarmuka pengguna (UI). Lewat UI tersebut, aplikasi mampu menarik data dummy secara online via HTTP request, yang kemudian secara otomatis tersimpan di dalam perangkat untuk mendukung akses offline. Data ini ditampilkan kepada pengguna dalam wujud list, lengkap dengan fasilitas untuk menghapus keseluruhan data jika diperlukan.

f. Code counter_provider.dart

```
import 'package:flutter/material.dart';

class CounterProvider with ChangeNotifier {
  int _counter = 0;

  int get counter => _counter;

  void increment() {
    _counter++;
    notifyListeners();
  }

  void decrement() {
    if (_counter > 0) {
      _counter--;
      notifyListeners();
    }
  }
}
```

g. Code counter_screen.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'counter_provider.dart';

class CounterScreen extends StatelessWidget {
  const CounterScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text("Ini adalah aplikasi Counter"),
        backgroundColor: Colors.red,
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Text("Isi Counter: "),
            Consumer<CounterProvider>(
              builder: (context, counter, child) {
                return Text(
                  "${counter.counter}",
                  style: const TextStyle(fontSize: 40,
fontWeight: FontWeight.bold),
                );
              },
            ),
          ],
        ),
      ),
      floatingActionButton: Column(
        mainAxisAlignment: MainAxisAlignment.end,
        children: [
          FloatingActionButton(
            heroTag: 'btn1',
```

```
            onPressed: () {
              Provider.of<CounterProvider>(context, listen:
false).increment();
            },
            child: const Icon(Icons.add),
          ),
          const SizedBox(height: 10),
          FloatingActionButton(
            heroTag: 'btn2',
            onPressed: () {
              Provider.of<CounterProvider>(context, listen:
false).decrement();
            },
            child: const Icon(Icons.remove),
          )
        ],
      ),
    );
  }
}
```

h. Code main.dart (Versi 2 - Gabungan SQLite & Provider)

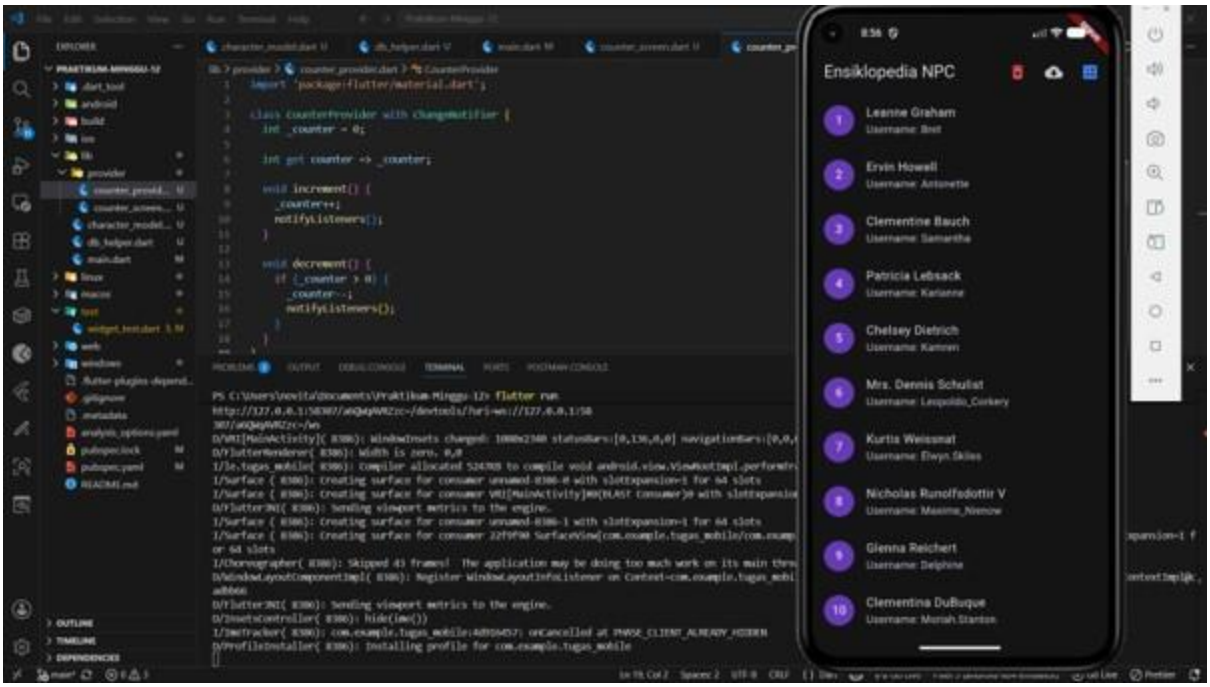
```
// Nama: Wafiq Nur Azizah
// NIM: 2311102270

import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'package:provider/provider.dart';

import 'character_model.dart';
import 'db_helper.dart';
import 'counter_provider.dart';
import 'counter_screen.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => CounterProvider(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Gabungan SQLite & Provider',
      theme: ThemeData.dark(useMaterial3: true),
      home: const CharacterListScreen(),
    );
  }
}
```

```
class CharacterListScreen extends StatefulWidget {
  const CharacterListScreen({super.key});

  @override
  State<CharacterListScreen> createState() =>
CharacterListScreenState();
}

class CharacterListScreenState extends
State<CharacterListScreen> {
  List<Character> _characters = [];
  bool _isLoading = false;

  @override
  void initState() {
    super.initState();
    _loadFromLocal();
  }

  Future<void> _loadFromLocal() async {
    setState(() => _isLoading = true);
    final localData = await
DatabaseHelper.instance.getAllCharacters();
    setState(() {
      _characters = localData;
      _isLoading = false;
    });
  }

  Future<void> _deleteAllData() async {
    bool? confirm = await showDialog<bool>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('Hapus Semua Data?'),
        content: const Text('apakah anda yakin?'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context, false),
            child: const Text('Batal'),
          ),
          TextButton(
            onPressed: () => Navigator.pop(context, true),
            child: const Text('Hapus', style: TextStyle(color:
Colors.red)),
          ),
        ],
      ),
    );

    if (confirm != true) return;

    setState(() => _isLoading = true);
    try {
      await DatabaseHelper.instance.clearTable();
      setState(() {
        _characters.clear();
      });
```

```
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Semua data berhasil
dihapus.')),
        );
      }
    } catch (e) {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text('Terjadi kesalahan: $e')),
        );
      }
    } finally {
      setState(() => _isLoading = false);
    }
  }

  Future<void> _fetchFromAPI() async {
    setState(() => _isLoading = true);
    try {
      final response = await http.get(
        Uri.parse('https://jsonplaceholder.typicode.com/users'),
      );

      if (response.statusCode == 200) {
        final List<dynamic> dataJson =
json.decode(response.body);
        await DatabaseHelper.instance.clearTable();

        List<Character> freshCharacters = [];
        for (var item in dataJson) {
          final char = Character.fromMap(item);
          freshCharacters.add(char);
          await DatabaseHelper.instance.insertCharacter(char);
        }
        setState(() => _characters = freshCharacters);

        if (mounted) {
          ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('Berhasil sync data
dari server!')),
          );
        }
      }
    } catch (e) {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Gagal konek internet,
pastikan online.')),
        );
      }
    } finally {
      setState(() => _isLoading = false);
    }
  }

  @override
```

```
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Ensiklopedia NPC'),
        actions: [
          IconButton(
            icon: const Icon(Icons.delete_forever, color:
Colors.redAccent),
            tooltip: 'Hapus Semua Data',
            onPressed: _characters.isEmpty ? null :
_deleteAllData,
          ),
          IconButton(
            icon: const Icon(Icons.cloud_download),
            tooltip: 'Sync dari Server',
            onPressed: _fetchFromAPI,
          ),
          IconButton(
            icon: const Icon(Icons.calculate, color:
Colors.blueAccent),
            tooltip: 'Buka Counter Provider',
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(builder: (context) => const
CounterScreen()),
              );
            },
          ),
        ],
      ),
      body: _isLoading
          ? const Center(child: CircularProgressIndicator())
          : _characters.isEmpty
              ? const Center(child: Text('Data kosong.'))
              : ListView.builder(
                  itemCount: _characters.length,
                  itemBuilder: (context, index) {
                    final char = _characters[index];
                    return ListTile(
                      leading: CircleAvatar(
                        backgroundColor: Colors.deepPurple,
                        child: Text(char.id.toString()),
                      ),
                      title: Text(char.name),
                      subtitle: Text('Username:
${char.username}'),
                    );
                  },
                ),
    );
  }
}
```

i. Screenshot output



j. Penjelasan dari code dan output (korelasi)

Dari code di atas ini menunjukkan bagaimana manajemen state reaktif berbasis Provider diintegrasikan ke dalam aplikasi Flutter, dengan tetap mempertahankan fitur sinkronisasi API dan database lokal (SQLite). Dalam penerapannya, CounterProvider berfungsi sebagai pusat logika state yang akan memperbarui UI secara otomatis lewat pemanggilan notifyListeners(). Di sisi lain, CounterScreen hadir sebagai antarmuka khusus yang memanfaatkan widget Consumer untuk merender pembaruan data secara efisien, serta menyediakan FloatingActionButton untuk interaksi pengguna. Integrasi akhir dilakukan pada main.dart, di mana keseluruhan aplikasi dibungkus oleh ChangeNotifierProvider pada level root. Untuk menjembatani kedua fungsionalitas tersebut, sebuah tombol navigasi dengan Navigator.push disematkan pada AppBar "Ensiklopedia NPC", memungkinkan transisi halaman yang mulus tanpa mengganggu sistem yang sudah berjalan.